

# AN13769

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

Rev. 0 — 13 December 2022

Application note

### Document information

| Information | Content                                                                                                                                           |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Keywords    | AN13769, TensorFlow Lite, DeepViewRT, PyTorch, Machine Learning (ML), ONNX, NPU, NCHW, NHWC                                                       |
| Abstract    | This document focuses on the conversion, quantization, and optimization of PyTorch models to the TensorFlow Lite and DeepViewRT inference engines |



1 Introduction

Machine learning (ML) frameworks such as TensorFlow, PyTorch, ONNX, and others are available for developing ML models.

NXP provides easy-to-use tools to perform quantization and conversion to TensorFlow Lite (\*.tflite) or NXP proprietary inference engine, DeepViewRT (\*.rtm). This allows users to bring their own model and convert it to NXP supported optimized inference engines to run on NXP SoC. For more information about these tools and how to install them, see the [eIQ ML Software Development Environment](#).

Table 1. NXP eIQ supported inference engines for NPU

| NXP eIQ inference engines | i.MX 8M Plus | i.MX 93 |
|---------------------------|--------------|---------|
| TensorFlow Lite           | ✓            | ✓       |
| DeepViewRT                | ✓            | NA      |

This document focuses on the quantization, optimization, and conversion of PyTorch models to either TensorFlow Lite (\*.tflite) or DeepViewRT (\*.rtm). Apart from PyTorch, the steps and tools included in this document may be used to convert any other ONNX model required to run on NXP NPUs, regardless of the ML framework used to create it.

2 Open Neural Network Exchange (ONNX)

Open Neural Network Exchange (ONNX) is an open standard for ML interoperability and its main purpose is to allow the use of multiple ML frameworks during any project development. By using ONNX, developers can use one framework for a specific phase of a project and then, if necessary, move to another framework that may be more desirable for the next phase. The option to select more than one framework for development can be helpful and provides the developer access to a wider set of tools. For more information about ONNX format, see [Open Neural Network Exchange](#).

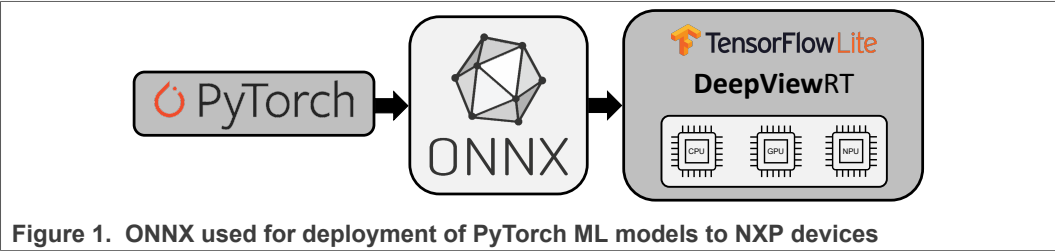


Figure 1. ONNX used for deployment of PyTorch ML models to NXP devices

3 Export PyTorch models to ONNX format

This document assumes that the user already has knowledge on PyTorch framework and Python programming language. Steps required to export PyTorch models to ONNX format and to perform post-training quantization are included in this application note. For more information on PyTorch and ONNX, refer to their open-source communities.

NXP "eIQ Model Tool" can be used to quantize and convert exported ONNX models from PyTorch framework to TensorFlowLite (\*.tflite) or DeepViewRT (\*.rtm). The first step is to create and train a PyTorch model, or else download a pre-trained one. The model is exported to 32 floating-point ONNX format using a Python script. The steps mentioned below assume that **PyTorch model is not quantized**.

### 3.1 Load the PyTorch model

To create or load a PyTorch model, three options are available for the developer.

- **Option 1:** Use your own PyTorch model:

If the model is created and trained from scratch using PyTorch framework, the same script can be used. Instantiate the model and set it to evaluation mode (`model.eval()`). See [Section 3.2](#) for exporting model to ONNX format.

- **Option 2:** Load a pre-trained PyTorch model (\*.pt/pth).

There are two methods to load pre-trained PyTorch models which are as follows:

1. Load the entire model. To load the entire PyTorch model, use the example snippet as follows:

```
model = torch.load(PATH)
model.eval()
```

If this method is used, make sure to load the entire model and not only the learned parameters of the trained model. Even though this method is the easiest and fastest way to load a model, this approach is not recommended since code may break in various ways.

2. Load only the learned parameters of the trained model. Make sure to load only the learned parameters by using the following code:

```
model = TheModelClass(*args, **kwargs) #This is the model architecture
model.load_state_dict(torch.load(PATH))
model.eval()
```

This method is more common and safer to perform. However, user must have the architecture implementation of the model, otherwise it is not possible to load the learned parameters. This method requires more effort.

- **Option 3:** Use the PyTorch Hub or PyTorch packages such as 'torchvision':

PyTorch Hub is a repository of pre-trained, popular, ready-to-use PyTorch models available for the community. Follow the documentation of the respective model. Also, pre-trained models can be obtained from the 'torchvision' package. Some examples are as follows:

```
import torch
# Load the Wide-ResNet50-2 Model
from torchvision.models import wide_resnet50_2, Wide_ResNet50_2_Weights
weights = Wide_ResNet50_2_Weights.IMAGENET1K_V2
model = wide_resnet50_2(weights=weights)
model.eval()

# Load the ResNet50 Model
from torchvision.models import resnet50, ResNet50_Weights
model = resnet50(weights=ResNet50_Weights.DEFAULT)
model.eval()
```

**Note:**

1. `model.eval()` must be called in all the three options before exporting to ONNX format otherwise failure might occur.
2. There are other methods to load PyTorch models from PyTorch Hub. See the documentation in the PyTorch community.

### 3.2 Export the model to ONNX format

Once the PyTorch model has been successfully loaded, it can be exported to ONNX format by using the following code. If no errors are encountered during export, user can have the 32 floating-point ONNX model ready for post-training quantization using the

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

"eIQ Model Tool". Code example is for a model with single input and single output. For models with more inputs/outputs, the code has to be modified.

```
# Export model to ONNX format
image_height = 224
image_width = 224
dummy_input = torch.zeros(1, 3, image_height, image_width) #BCHW
torch.onnx.export(model,
                  dummy_input,
                  'my_model.onnx',
                  verbose=False, opset_version=15,
                  training=torch.onnx.TrainingMode.EVAL,
                  do_constant_folding=True,
                  input_names=['input'],
                  output_names=['output'],
                  dynamic_axes=None)
```

**Note:** Our BSP release LF5.15.52\_2.1.0 currently supports ONNX opset version <= 15, however, some models are not entirely compatible with this version. If user runs into issues, it is recommended to experiment with this number. Raising this number typically improves performance since more operators are supported.

There are some models that output the following message when exported to ONNX format. Tests have shown that this warning occurs when using ONNX opset version 13 or above. If below warning appears, downgrade to ONNX opset version 12 or earlier until this message stops appearing.

**WARNING:** The shape inference of prim::Constant type is missing, so it may result in wrong shape inference for the exported graph. Please consider adding it in symbolic function.

### 3.3 Select the type of conversion

The converted ONNX model can be executed on i.MX NPUs by choosing one of three paths:

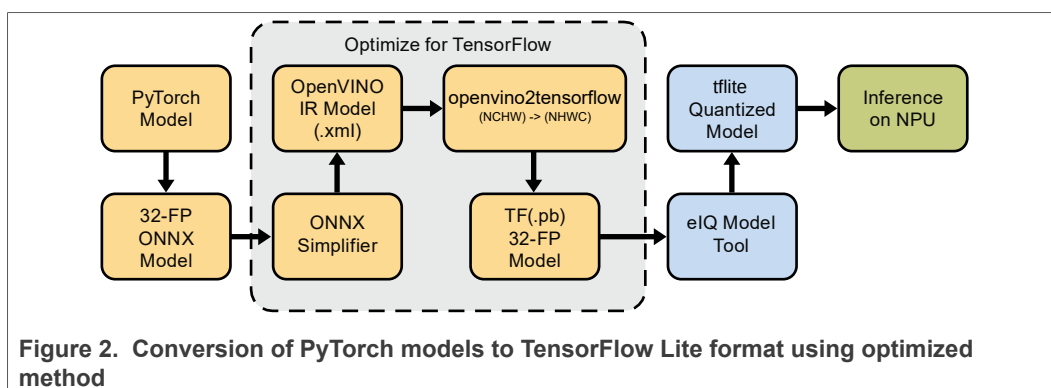
- Convert from ONNX to TensorFlow Lite (\*.tflite): Fully optimized for NHWC data layout.
- Convert from ONNX to TensorFlow Lite (\*.tflite): Not fully optimized, direct conversion.
- Convert from ONNX to DeepViewRT (\*.rtm).

When converting ONNX models that runs on NXP devices, it is recommended to use the "eIQ Model Tool".

Keep in mind that ONNX models cannot be quantized unless they are opset version 10 or higher. If not, re-export from original framework with a higher ONNX opset version.

For more information, see the *eIQ Toolkit User's Guide* (document [EIQTUG](#)).

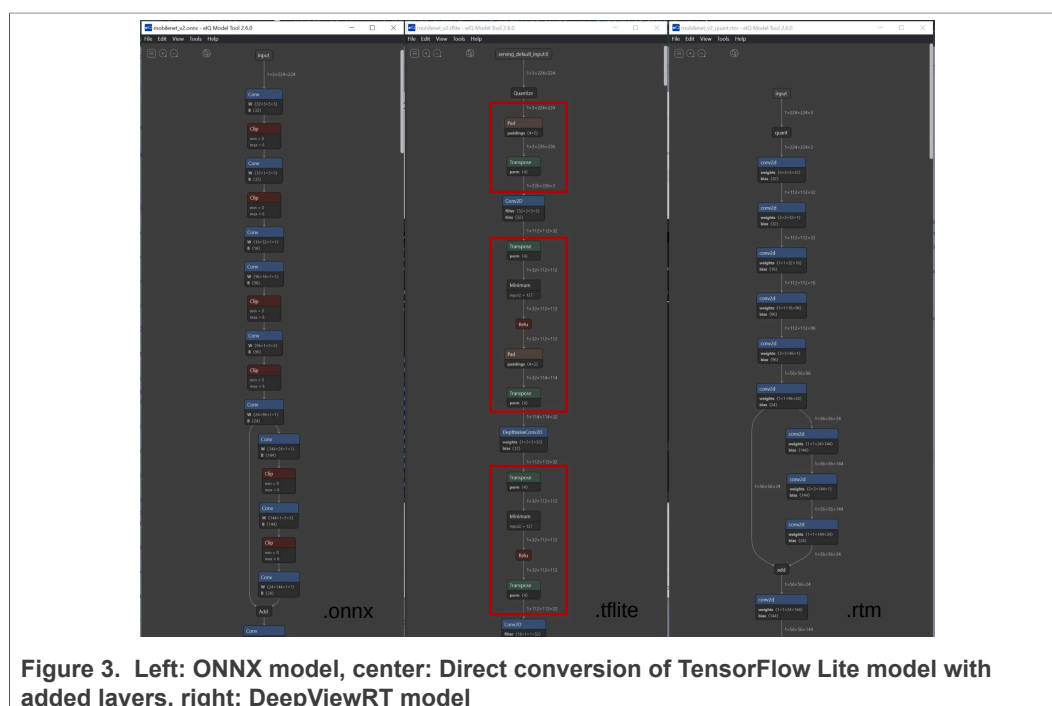
## 4 Convert from ONNX to TensorFlow Lite (NHWC data layout)



## 4.1 Workaround for NCHW to NHWC issue in TensorFlow Lite models

A known issue exists when converting ONNX models generated with PyTorch to TensorFlow Lite format. Conversion adds extra transpose layers. These added extra layers are to transpose the tensors from NCHW [N: batch, C: channels, H: height, W: width] data layout to NHWC. This happens because PyTorch uses the NCHW data layout and TensorFlow only supports NHWC. As an example, [Figure 3](#) shows a model that is converted using direct method. In this example, additional transpose layers are added to the model increasing the inference time from 3.61 ms to 13.20 ms on the i.MX 8M Plus. These added layers can increase the inference time since the data is transposed on the fly. This can be avoided by transposing to the correct layout during model conversion. This document includes the steps for a workaround that uses offline Open-Source Software (OSS) conversion tools.

**Note:** Make sure to use the OSS conversion tools to fix the NHWC data layout, otherwise performance of TensorFlow Lite model is degraded.



## 4.2 Install the OSS tools

The tool required to fix the NHWC data layout is called "openvino2tensorflow". This tool is an open-source project available at [openvino2tensorflow](https://github.com/openvino2tensorflow/openvino2tensorflow). It is licensed under the MIT License and converts ONNX models to TensorFlow (\*.pb) saved models with the NHWC data layout. Install this tool in the host machine and ensure to use the docker container. All steps included in this document are done using the docker container installation. Once installed, continue with the steps as follows.

### 4.2.1 Convert from ONNX to TensorFlow Lite

1. Run the "openvino2tensorflow" docker container:

```
$ sudo docker run -it --rm -v `pwd`: /home/user/workdir ghcr.io/pinto0309/
openvino2tensorflow:latest
```

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

2. Navigate to the root of the docker container and simplify the ONNX model using "ONNX Simplifier tool". Some models may already be in their most simplified form, but still run this command to make sure:

```
$ cd
$ onnxsim /input/path/model.onnx /output/path/model.onnx
```

3. Convert the ONNX simplified model to an \*.xml file representation using the model optimizer `mo` command. Some models may require that the input data is normalized with mean and standard deviation, but do not include these parameters when converting. Internal tests show that doing this may affect accuracy of model. To ensure that the `mo` command is recognized, make sure that user is in the root folder of the docker container.

```
$ mo --input_model model.onnx
```

4. Use the "openvino2tensorflow" tool to convert to TensorFlow (\*.pb) saved model. This changes the NCHW data layout to NHWC data layout, the one that TensorFlow Lite supports. Keep in mind that some complex models may require some extra effort if the tool is not able to convert automatically all tensors to the correct data layout. If this happens, see the documentation of the tool to fix it manually.

```
$ openvino2tensorflow --model_path model.xml --model_output_path model --
output_saved_model --output_pb --non_verbose
```

5. The generated folder has the 32 floating-point TF (\*.pb) model which now can be loaded to the "eIQ Model Tool" for post-training-quantization and conversion to TensorFlow Lite format.
6. Open "eIQ Model Tool" by clicking the **Model Tool** button inside "eIQ Portal" as shown in [Figure 4](#).
7. Load the 32 floating-point `saved_model.pb` model. One way to load this model is to drag it to the "eIQ Model Tool" window, or click **Open Model...** button. Wait for it to be loaded.
8. When model is loaded, click **File > Convert > TensorFlow Lite (.tflite)**. A window is shown with quantization and export options.
9. Enable quantization by selecting this option in **Quantization Settings**. For i.MX 8M Plus NPU it is recommended to select **Per Tensor** quantization, but **Per Channel** can also be used, see [Section 8](#). Also, input data type and output data type can be changed if desired.
10. Load the calibration dataset. This dataset is necessary to load, otherwise accuracy of the model drops during quantization process and negatively impacts the deployment of model. Using around 100 to 500 images is enough for most models, but do experiment with this quantity, as it is one type of hyperparameter.
11. Click **Convert** and wait for the quantization and export processes to finish. Once finished, a window pops up with the converted and quantized TensorFlow Lite model, which is now optimized for the NPU inference acceleration.

Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

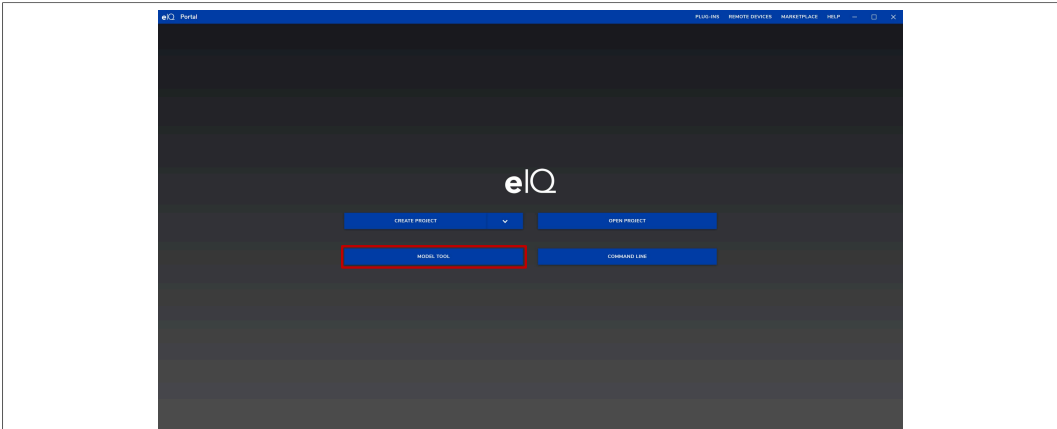


Figure 4. Open eIQ Model Tool from eIQ Portal

Figure 5 shows the comparison between TensorFlow Lite model converted without modifying the data layout and the TensorFlow Lite model converted with the OSS tool and has a fixed NHWC data layout. The fixed model performs much better since less operation is performed during inference time.

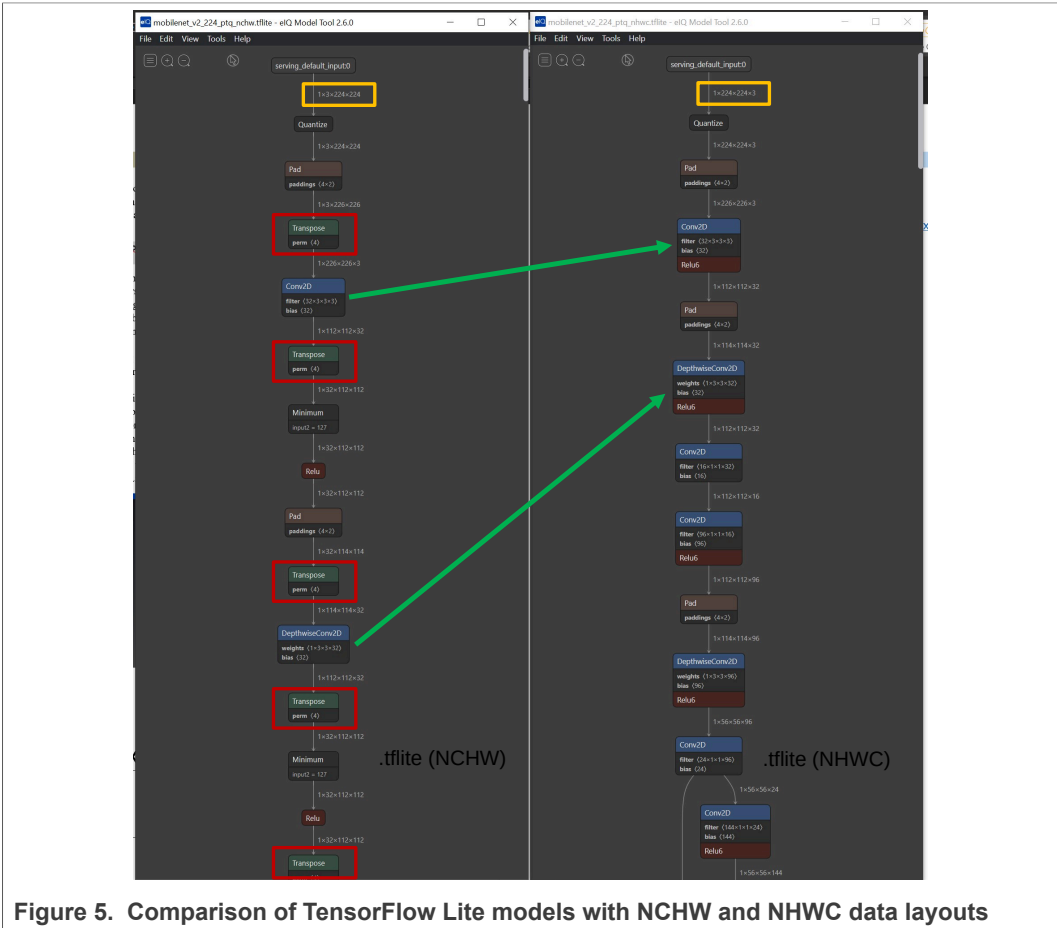
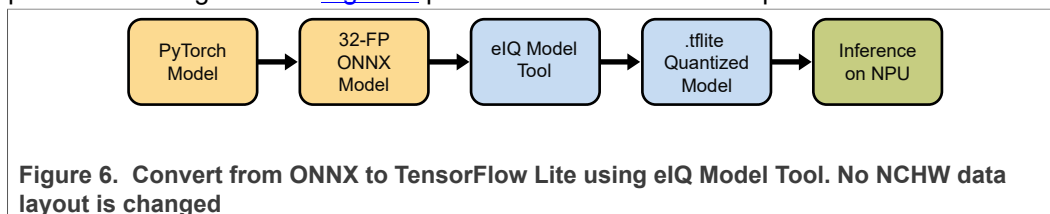


Figure 5. Comparison of TensorFlow Lite models with NCHW and NHWC data layouts

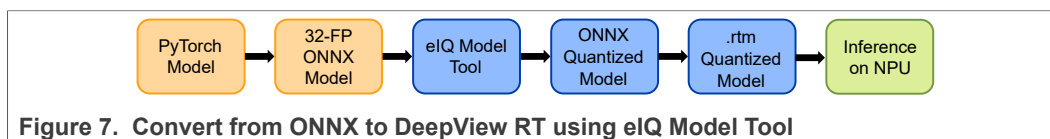
## 5 Convert from ONNX to TensorFlow Lite (direct conversion)

Currently "eIQ Model Tool" does not support a complete data layout conversion for the TensorFlow Lite format. The following approach can be used if desired but might have performance degradation. [Figure 6](#) provides an overview of this process.



1. Open "eIQ Model Tool" by clicking the **Model Tool** button inside "eIQ Portal" as shown in [Figure 4](#).
2. Load the ONNX model. One way to load the model is to drag the ONNX model to the "eIQ Model Tool" window, or click **Open Model...** button. Wait for the ONNX model to be loaded. Make sure that PyTorch framework has not quantized this model previously.
3. When model is loaded, click **File > Convert > TensorFlow Lite (.tflite)**. A window opens with quantization and export options.
4. Enable quantization by selecting this option in **Quantization Settings**. For i.MX 8M Plus NPU it is recommended to select **Per Tensor** quantization, but **Per Channel** can also be used, see [Section 8](#). Also, input data type and output data type can be changed if desired.
5. Load the calibration dataset. This is necessary, otherwise accuracy of model drops during quantization process and negatively impacts the deployment of model. Using around 100 to 500 images is enough for most models, but do experiment with this quantity, as this is one type of hyperparameter.
6. Click **Convert** and wait for the quantization and export processes to finish. A window pops up with the converted and quantized TensorFlow Lite model. This model runs on the NPU, but performance is worse compared to a TensorFlow Lite model with NHWC data layout.

## 6 Convert from ONNX to DeepViewRT



1. Open "eIQ Model Tool" by clicking **Model Tool** button inside "eIQ Portal" as shown in [Figure 4](#).
2. Load the ONNX model. One way to load this model is to drag the ONNX model to the "eIQ Model Tool" window, or click **Open Model...** button. Wait for the ONNX model to be loaded. Make sure that PyTorch framework has not quantized this model previously.
3. When model is loaded, click **File > Convert > ONNX (\*.onnx)**. A window is shown with quantization and export options.
4. Enable quantization by selecting this option in **Quantization Settings**. For i.MX 8M Plus NPU it is recommended to select **Per Tensor** quantization, but **Per Channel** can also be used, see [Section 8](#).

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

5. Load the calibration dataset. This is necessary, otherwise accuracy of the model drops during quantization process and negatively impacts the deployment of model. Using around 100 to 500 images is enough for most models, but do experiment with this quantity, as this is one type of hyperparameter.

**Note:** If an error message as shown in [Figure 8](#) occurs, most probably there is a monochromatic image (just one channel, no RGB) in the calibration dataset. Replace this image with a 3-channel, RGB image. This issue is not related to ONNX model.

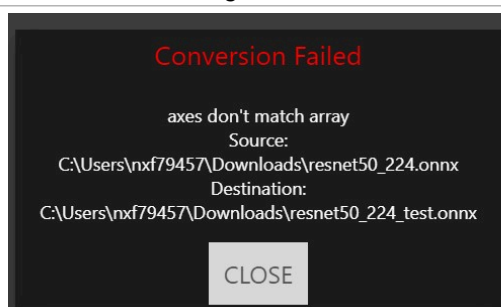


Figure 8. Error during quantization if using monochromatic images for calibration

6. Click **Convert** and wait for the quantization and export processes to finish, a window pops up with the converted and quantized model. Consider [Figure 9](#) as an example.

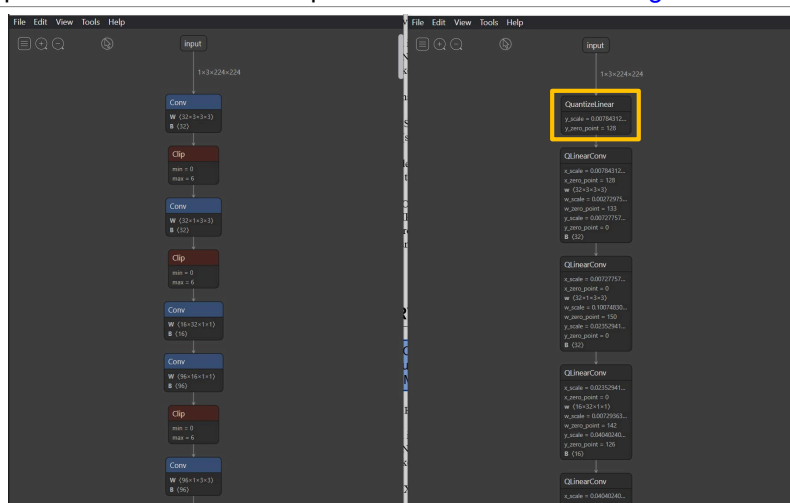


Figure 9. Left: 32 floating-point ONNX model, right: quantized ONNX model

7. Finally, convert the pre-quantized ONNX model by clicking **File > Convert > DeepViewRT (.rtm)**. A window pops up with quantization and export options. Also, the labels.txt file can be embedded into the model. Clicking **Convert** generates the quantized, NPU-optimized model for DeepViewRT engine.

## 7 Considerations for model conversion

Things to keep in mind when converting models are as follows:

- Ensure to quantize the model using **Per-Tensor** quantization for the i.MX 8M Plus NPU, which performs better with this type of quantization.
- If creating and training a model from scratch in PyTorch, make sure that the output tensors are normalized for quantization to be successful. A low accuracy quantized model or complete failure can result from doing otherwise.

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

- Some ONNX models might fail to convert to DeepViewRT (\*.rtm) and TensorFlow Lite (\*.tflite) formats. Try experimenting with the ONNX opset version and see if failure is related to an unsupported operator.
- For better performance, use the OSS tools to optimize the ONNX conversion to TensorFlow Lite model with NHWC data layout.

## 8 Benchmarks of PyTorch models

The models included in [Table 2](#) have been downloaded from PyTorch Hub and converted to ONNX format. These models are then converted to TensorFlow Lite and DeepViewRT formats with the steps explained above.

**Note:** Direct conversion from ONNX to TensorFlow Lite keeps the NCHW data layout and affect the inference performance of the models. For some models this performance degradation may affect too much, as seen for the 'mobilenet\_v2' model.

All models in [Table 2](#) use ONNX Opset version 15.

**Table 2. Benchmarks of PyTorch models converted to TensorFlow Lite and DeepViewRT on i.MX 8M Plus EVK**

| Model                               | Size      | Arm Cortex-A53<br>(four threads) | NPU      |
|-------------------------------------|-----------|----------------------------------|----------|
| wide_resnet50_2_224_ptq_nhwc.rtm    | 224x224x3 | 2501.74 ms                       | 23.77 ms |
| wide_resnet50_2_224_ptq_nhwc.tflite | 224x224x3 | 672.85 ms                        | 24.38 ms |
| wide_resnet50_2_224_pcq_nhwc.tflite | 224x224x3 | 673.07 ms                        | 34.98 ms |
| wide_resnet50_2_224_ptq_nchw.tflite | 3x224x224 | 702.52 ms                        | 28.87 ms |
| mobilenet_v2_224_ptq_nhwc.rtm       | 224x224x3 | 123.63 ms                        | 3.00 ms  |
| mobilenet_v2_224_ptq_nhwc.tflite    | 224x224x3 | 58.13 ms                         | 3.61 ms  |
| mobilenet_v2_224_pcq_nhwc.tflite    | 224x224x3 | 58.18 ms                         | 7.72 ms  |
| mobilenet_v2_224_ptq_nchw.tflite    | 3x224x224 | 180.21 ms                        | 13.20 ms |
| resnet50_224_ptq_nhwc.rtm           | 224x224x3 | 1120.43 ms                       | 12.17 ms |
| resnet50_224_ptq_nhwc.tflite        | 224x224x3 | 278.76 ms                        | 12.76 ms |
| resnet50_224_pcq_nhwc.tflite        | 224x224x3 | 278.81 ms                        | 20.33 ms |
| resnet50_224_ptq_nchw.tflite        | 3x224x224 | 294.90 ms                        | 16.64 ms |

## 9 ONNX-Runtime for ML inference of ONNX models

ONNX-Runtime is an open-source ML inference engine used to execute ONNX models across various deployment targets using a single set of API. To perform the ML inference, ONNX-Runtime uses a mechanism to delegate the inference execution to an underlying framework or hardware, which are called Execution Providers (EP).

By default, the ONNX-Runtime uses the CPU EP, which executes inference on the Cortex-A CPU with Arm Neon SIMD instructions. It also supports computation on the GPU and NPU for the i.MX 8M Plus, but are no longer maintained and may be deprecated in the future. For more information about ONNX release version in our BSP and supported EPs, see the *Machine Learning User's Guide* (document [IMXMLUG](#)). For more information about ONNX-Runtime, see the official GitHub repository or the official webpage for ONNX-Runtime.

### 9.1 ONNX-Runtime performance test

ONNX-Runtime provides its own tool to benchmark ONNX models, called "onnxruntime\_perf\_test". To use this tool, it is necessary to first generate test data in a specific format and structure. Once this data is generated, the ONNX model performance can be measured on the desired target, which in this case is the i.MX 8M Plus EVK. To use this tool, follow the steps below:

#### 9.1.1 Generate test data

- Clone the official ONNX-Runtime repository on the host machine:

```
$ git clone https://github.com/microsoft/onnxruntime.git
```

- Checkout to the same installed version of ONNX-Runtime (in this case v1.10.0):

```
$ cd onnxruntime
$ git checkout v1.10.0
```

- Navigate to Python tools folder inside the repository:

```
$ cd tools/python
```

- Create a Python script for the model to be tested. The *PythonTools.md* file, located in the same folder, has instructions on how to achieve this script.
- Once the script is created, run it, and wait for the data to be generated. A folder is created with the correct structure for the "onnxruntime\_perf\_test" tool.
- Copy the generated folder to the target device. For the i.MX 8M Plus EVK, copy the folder to this path: `/usr/bin/onnxruntime-1.10.0/`.

#### 9.1.2 Performance test

If the test data has been generated as explained in [Section 9.1.1](#), the model is already included inside the test data folder. If so, skip to step 2:

1. Copy the converted model to the target device. The easiest way is by using `scp` command. On the host machine, use the following command:

```
$ scp resnet50_224.onnx root@<ip-address>:/usr/bin/onnxruntime-1.10.0/
```

2. On the target, navigate to the `onnxruntime-1.10.0` folder:

```
$ cd /usr/bin/onnxruntime-1.10.0/
```

## Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

3. Perform the inference benchmark using the CPU EP (four threads are used):

```
$ ./onnxruntime_perf_test model/wide_resnet50_2_224_ptq.onnx -r 1 -e cpu

Session creation time cost: 4.97448 s
Total time cost (including warm-up): 1.44954 s
Total inference requests: 2
Warm-up inference time cost: 725.622 ms
Average inference time cost (excluding warm-up): 723.922 ms
Total inference run time: 0.723942 s
Avg CPU usage: 98 %
Peak working set size: 207081472 bytes
```

4. Perform the inference benchmark using the NPU with VSI\_NPU EP (deprecated):

```
$ ./onnxruntime_perf_test model/wide_resnet50_2_224_ptq.onnx -r 1 -e vsi_npu
Session creation time cost: 0.561183 s
Total time cost (including warm-up): 51.8483 s
Total inference requests: 2
Warm-up inference time cost: 51762.9 ms
Average inference time cost (excluding warm-up): 85.4016 ms
Total inference run time: 0.0854216 s
Avg CPU usage: 33 %
Peak working set size: 393310208 bytes
```

## 10 Conclusion

This document describes the steps used to convert PyTorch models to either TensorFlow Lite or DeepViewRT formats. eIQ inference engines officially support TensorFlow Lite and DeepViewRT models and leverage the full potential of the NPUs, providing optimal performance during inference process. Being able to convert ONNX models to these formats allows the developers to use a wider set of tools available in other ML frameworks, such as PyTorch, and deploy the models to NXP devices for inference acceleration at the edge.

If you have any queries, feel free to reach out to us at <https://community.nxp.com/>.

## 11 References

The following references are available to supplement this document. Some of the references listed below may be available only under a non-disclosure agreement (NDA). To request access to these references, contact your local NXP field applications engineer (FAE) or sales representative:

- eIQ ML Software Development Environment: <https://www.nxp.com/design/software/development-software/eiq-ml-development-environment:EIQ>
- GitHub repository for ONNX-Runtime: <https://github.com/microsoft/onnxruntime>. Licensed under *MIT License*.
- GitHub repository for ONNX Simplifier: <https://github.com/daquexian/onnx-simplifier>. Licensed under *Apache 2.0 License*.
- GitHub repository for openvino2tensorflow: [openvino2tensorflow](https://github.com/openvino2tensorflow). Licensed under *MIT License*.
- GitHub repository for OpenVINO: <https://github.com/openvinotoolkit/openvino>. Licensed under *Apache 2.0 License*.
- PyTorch: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- PyTorch Hub: <https://pytorch.org/hub/>
- PyTorch snippets obtained from the documentation available at: <https://pytorch.org/>. PyTorch has a *BSD-style license*, as found in the <https://github.com/pytorch/pytorch/blob/master/LICENSE> file.

- Models and pre-trained weights from the torchvision package: <https://pytorch.org/vision/stable/models.html>

## 12 Revision history

The [Table 3](#) lists the substantive changes done to this document since the initial release.

Table 3. Revision history

| Revision number | Date             | Substantive changes |
|-----------------|------------------|---------------------|
| 0               | 13 December 2022 | Initial release     |

## 13 Legal information

### 13.1 Definitions

**Draft** — A draft status on a document indicates that the content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included in a draft version of a document and shall have no liability for the consequences of use of such information.

### 13.2 Disclaimers

**Limited warranty and liability** — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

**Right to make changes** — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

**Suitability for use** — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

**Applications** — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification. Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

**Terms and conditions of commercial sale** — NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

**Export control** — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

**Suitability for use in non-automotive qualified products** — Unless this data sheet expressly states that this specific NXP Semiconductors product is automotive qualified, the product is not suitable for automotive use. It is neither qualified nor tested in accordance with automotive testing or application requirements. NXP Semiconductors accepts no liability for inclusion and/or use of non-automotive qualified products in automotive equipment or applications.

In the event that customer uses the product for design-in and use in automotive applications to automotive specifications and standards, customer (a) shall use the product without NXP Semiconductors' warranty of the product for such automotive applications, use and specifications, and (b) whenever customer uses the product for automotive applications beyond NXP Semiconductors' specifications such use shall be solely at customer's own risk, and (c) customer fully indemnifies NXP Semiconductors for any liability, damages or failed product claims resulting from customer design and use of the product for automotive applications beyond NXP Semiconductors' standard warranty and NXP Semiconductors' product specifications.

**Translations** — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

**Security** — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately. Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at [PSIRT@nxp.com](mailto:PSIRT@nxp.com)) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

### 13.3 Trademarks

Notice: All referenced brands, product names, service names, and trademarks are the property of their respective owners.

**NXP** — wordmark and logo are trademarks of NXP B.V.

**AMBA, Arm, Arm7, Arm7TDMI, Arm9, Arm11, Artisan, big.LITTLE, Cordio, CoreLink, CoreSight, Cortex, DesignStart, DynamiQ, Jazelle, Keil, Mali, Mbed, Mbed Enabled, NEON, POP, RealView, SecurCore, Socrates, Thumb, TrustZone, ULINK, ULINK2, ULINK-ME, ULINK-PLUS, ULINKpro, uVision, Versatile** — are trademarks or registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. The related technology may be protected by any or all of patents, copyrights, designs and trade secrets. All rights reserved.

**eiQ** — is a trademark of NXP B.V.

---

Deploy PyTorch Models to NXP Supported Inference Engines Using ONNX

i.MX — is a trademark of NXP B.V.

## Contents

---

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction .....</b>                                                 | <b>2</b>  |
| <b>2</b>  | <b>Open Neural Network Exchange (ONNX) .....</b>                          | <b>2</b>  |
| <b>3</b>  | <b>Export PyTorch models to ONNX format .....</b>                         | <b>2</b>  |
| 3.1       | Load the PyTorch model .....                                              | 3         |
| 3.2       | Export the model to ONNX format .....                                     | 3         |
| 3.3       | Select the type of conversion .....                                       | 4         |
| <b>4</b>  | <b>Convert from ONNX to TensorFlow Lite<br/>(NHWC data layout) .....</b>  | <b>4</b>  |
| 4.1       | Workaround for NCHW to NHWC issue in<br>TensorFlow Lite models .....      | 5         |
| 4.2       | Install the OSS tools .....                                               | 5         |
| 4.2.1     | Convert from ONNX to TensorFlow Lite .....                                | 5         |
| <b>5</b>  | <b>Convert from ONNX to TensorFlow Lite<br/>(direct conversion) .....</b> | <b>8</b>  |
| <b>6</b>  | <b>Convert from ONNX to DeepViewRT .....</b>                              | <b>8</b>  |
| <b>7</b>  | <b>Considerations for model conversion .....</b>                          | <b>9</b>  |
| <b>8</b>  | <b>Benchmarks of PyTorch models .....</b>                                 | <b>10</b> |
| <b>9</b>  | <b>ONNX-Runtime for ML inference of ONNX<br/>models .....</b>             | <b>11</b> |
| 9.1       | ONNX-Runtime performance test .....                                       | 11        |
| 9.1.1     | Generate test data .....                                                  | 11        |
| 9.1.2     | Performance test .....                                                    | 11        |
| <b>10</b> | <b>Conclusion .....</b>                                                   | <b>12</b> |
| <b>11</b> | <b>References .....</b>                                                   | <b>12</b> |
| <b>12</b> | <b>Revision history .....</b>                                             | <b>13</b> |
| <b>13</b> | <b>Legal information .....</b>                                            | <b>14</b> |

---

Please be aware that important notices concerning this document and the product(s) described herein, have been included in section 'Legal information'.

---